

Formal Languages and Compilers
Prof. Breveglieri, Morzenti, Agosta
Written exam: laboratory question

06/02/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the `Lance` language with the ability to perform multiple assignments on the same line.

In the unmodified `Lance` language, assignments can only specify one variable on the left side of the `=` token, and one expression on the right-hand side of the `=` token. In addition to this syntax, comma-separated lists of variables and comma-separated lists of expressions shall be allowed to appear on the left and right sides of the `=` token respectively.

For an assignment to be syntactically correct, the length of the two lists must be the same. Additionally, the list on the left of the `=` token should not include array elements. When this is not the case, the compilation shall fail with a syntax error.

The order of variables in the list matches the order of expressions. In other words, the first expression is assigned to the first variable, the second expression is assigned to the second variable, and so on. The order in which the assignments are made in the compiled code is unspecified. Hence, dependencies between two expressions can be disregarded.

The following code snippet exemplifies the usage of the multiple assignment feature. In the snippet, three variables (`a`, `b` and `c`) are assigned by the same statement to three different values determined by the expressions on the right-hand side.

```
int a, b, c, d;

d = 10;
a, b, c = 5, 44 / 6, d * 2;

write(a); // 5
write(b); // 7
write(c); // 20
```

Hint: Be careful to not introduce ambiguities in the grammar. Indeed, the case in which the list of variables and expressions are of length one is already handled by the standard assignment operator grammar. Therefore, if in your grammar the lists can be of length one, *and no other modifications are made to the pre-existing rules*, the grammar will be ambiguous.

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (4 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

4. Given the following Lance code snippet:

```
int a[5], c=7;  
int d;
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the var_declarations nonterminal*. (5 points)

5. (**Bonus**) Discuss how you would modify your solution to the previous exercises in order to add support for assignments to array elements within a multiple assignment statement.